SEMAINE 12

Modèles de diffusion

Processus direct/inverse · DDPM · génération progressive · multi-modalité

S12 — OBJECTIFS (CE QUE VOUS SAUREZ FAIRE À LA FIN)

1. Expliquer l'intuition d'un **processus direct** qui transforme progressivement les données en bruit.
2. Décrire le **processus inverse** appris pour reconstruire des données à partir du bruit.
3. Comprendre les équations fondamentales du **DDPM** et pourquoi l'entraînement revient à prédire le bruit ajouté.
4. Expliquer la **génération progressive** et pourquoi l'échantillonnage est plus lent que dans un GAN ou un flow.
5. Situer DDIM et quelques stratégies d'accélération.
6. Comprendre le rôle du **schedule de bruit** β_t et de la variance inverse σ_t^2.
7. Expliquer le principe de la **classifier-free guidance** pour la génération conditionnelle.
8. Situer la **diffusion latente** comme variante efficace pour les images haute résolution, sans anticiper l'étude détaillée des modèles hybrides de S14.

MISE EN CONTEXTE — DE S11 À S12

En S11, nous avons appris à estimer un ***score*** $\nabla_x \log p(x)$ *et à utiliser ce champ de vecteurs pour guider un échantillonnage de type Langevin. La difficulté était que le score est mal estimé dans les régions de faible densité. La diffusion répond à cette difficulté en introduisant* ***plusieurs niveaux de bruit*** *: on apprend progressivement à revenir du bruit vers les données.*

QUESTION CENTRALE DE LA SEMAINE

Comment apprendre à transformer progressivement du bruit aléatoire en une donnée réaliste ?

LIEN AVEC LE PLAN DU COURS

La S12 est centrée sur les quatre éléments prévus au plan : **processus direct/inverse**, **DDPM**, **génération progressive** et **généralisation multi-modalité**. Les notions d'accélération sont présentées comme prolongement pratique, sans déplacer le cœur de la séance.

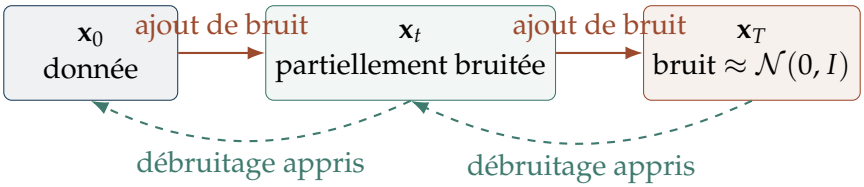
1 Introduction : pourquoi les modèles de diffusion ?

Les modèles de diffusion partent d'une idée simple : il est facile de **détruire progressivement une structure** en ajoutant du bruit. Si l'on apprend le chemin inverse, on peut partir d'un bruit gaussien et reconstruire progressivement une donnée plausible.

Dans le contenu de cette séance, les modèles de diffusion sont motivés par quatre propriétés : qualité d'échantillonnage souvent élevée, couverture des modes généralement favorable, entraînement souvent stable et fondements probabilistes solides. Ils évitent notamment le jeu antagoniste propre aux GAN.

INTUITION PHYSIQUE

Un processus direct transforme une donnée structurée en bruit. Le modèle apprend ensuite un processus inverse de **débruitage progressif**. La génération consiste donc à répéter beaucoup de petites corrections plutôt qu'à produire l'échantillon en une seule étape.



2 Processus direct : transformer les données en bruit

2.1 Chaîne de Markov discrète

On considère une séquence $\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_T$ de même dimension. À chaque étape, on ajoute une petite quantité de bruit gaussien :

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}\left(\mathbf{x}_t; \sqrt{1-\beta_t}\,\mathbf{x}_{t-1}, \beta_t I\right).$$

Le paramètre $\beta_t \in (0,1)$ contrôle la quantité de bruit ajoutée à l'étape t.

NOTATION ESSENTIELLE

On pose

$$\alpha_t = 1 - \beta_t, \qquad \bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s.$$

Quand t augmente, $\bar{\alpha}_t$ diminue : le signal d'origine s'affaiblit et le bruit devient dominant.

CHOISIR LE SCHEDULE DE β_t

Le **schedule** fixe la manière dont le bruit augmente avec t. Dans le DDPM original, un schedule linéaire de petites valeurs de β_t est utilisé. Une alternative très répandue est le **schedule cosinus**, qui fait décroître $\bar{\alpha}_t$ plus progressivement et préserve davantage le signal aux premiers instants.

À retenir : le schedule n'est pas un détail purement numérique : il contrôle le rapport signal/-bruit rencontré pendant l'entraînement et influence donc la difficulté du débruitage.

2.2 Échantillonner directement $\mathbf{x}_t$ depuis $\mathbf{x}_0$

Une propriété fondamentale permet d'éviter de simuler toutes les étapes intermédiaires :

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}\left(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0, (1-\bar{\alpha}_t) I\right).$$

Donc

$$\boxed{\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(0, I).}$$

POURQUOI PEUT-ON SAUTER DIRECTEMENT DE x_0 À x_t ?

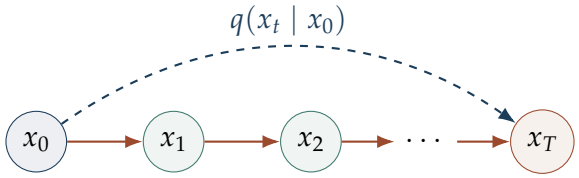
La composition de transitions gaussiennes reste gaussienne. En développant récursivement $q(x_t \mid x_{t-1})$ puis en regroupant les termes en x_0 et les bruits indépendants, on obtient exactement la moyenne $\sqrt{\bar{\alpha}_t} x_0$ et la variance $(1-\bar{\alpha}_t) I$.

Dans ce cours, on **admet la dérivation algébrique complète** et on retient surtout cette propriété, essentielle pour entraîner le modèle à un instant t choisi aléatoirement sans simuler toutes les étapes précédentes.

DU CONCEPT AU CODE — BRUITER DIRECTEMENT UNE DONNÉE

```
eps = torch.randn_like(x0)
xt = sqrt_alpha_bar[t] * x0 + sqrt_one_minus_alpha_bar[t] * eps
```

À observer : l'entraînement peut choisir n'importe quel instant t sans simuler $1, 2, \ldots, t-1$.



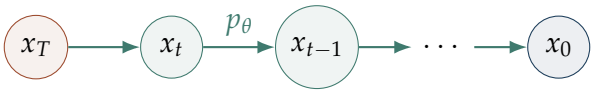
3 Processus inverse : apprendre à débruiter

Pour générer, on voudrait inverser la chaîne. Le vrai $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ dépend de la distribution des données et n'est pas directement disponible. On apprend donc une approximation paramétrée :

$$p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \Sigma_\theta(\mathbf{x}_t, t)) \,.$$

LE PROBLÈME ET LA SOLUTION

Problème : le processus direct est connu, mais son inverse dépend de la structure inconnue des données.
Solution : entraîner un réseau à prédire l'information nécessaire pour retirer progressivement le bruit.



débruitage progressif

4 DDPM : Denoising Diffusion Probabilistic Models

4.1 Que prédit réellement le réseau ?

Dans un DDPM, le réseau ne prédit généralement pas directement x_{t-1}. Il reçoit (x_t, t) et apprend à prédire le bruit ϵ qui a servi à produire x_t.

DE L'ELBO À LA PERTE SIMPLE — DÉRIVATION ADMISE

L'apprentissage probabiliste du DDPM peut être formulé par une **borne variationnelle** sur $-\log p_\theta(x_0)$. Cette borne se décompose en termes de divergence KL entre les transitions inverses vraies du processus direct et les transitions apprises $p_\theta(x_{t-1} \mid x_t)$.

Avec une variance inverse fixée et la paramétrisation de la moyenne par la prédiction du bruit $\epsilon_\theta(x_t, t)$, ces termes deviennent, à des poids dépendant de t près, des erreurs quadratiques entre le bruit réel ϵ et le bruit prédit. Le DDPM original montre qu'une version simplifiée non pondérée fonctionne particulièrement bien en pratique.

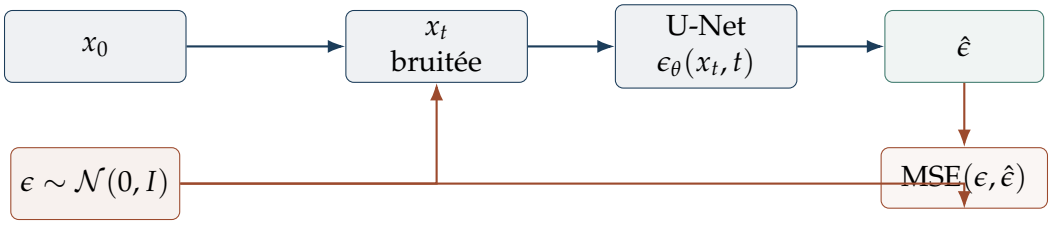
On **admet ici la dérivation complète ELBO → KL par pas → MSE sur le bruit** ; l'objectif pédagogique est de comprendre pourquoi la prédiction de ϵ constitue une paramétrisation naturelle du processus inverse.

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{t,\mathbf{x}_0,\boldsymbol{\epsilon}}\left[\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta\left(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t\right)\right\|^2\right].$$

LECTURE EN MOTS

On choisit une vraie donnée x_0, un instant t et un bruit ϵ. On construit x_t, puis on demande au réseau de retrouver le bruit qui a été ajouté. L'apprentissage devient donc une simple régression

MSE sur le bruit.



DU CONCEPT AU CODE — UNE ÉTAPE D'ENTRAÎNEMENT DDPM

```
t = torch.randint(0, T, (x0.size(0),), device=x0.device)
eps = torch.randn_like(x0)
xt = q_sample(x0, t, eps)
loss = F.mse_loss(model(xt, t), eps)
```

À observer : le modèle reçoit uniquement x_t et t ; la cible supervisée est le bruit exact ϵ utilisé pour créer x_t.

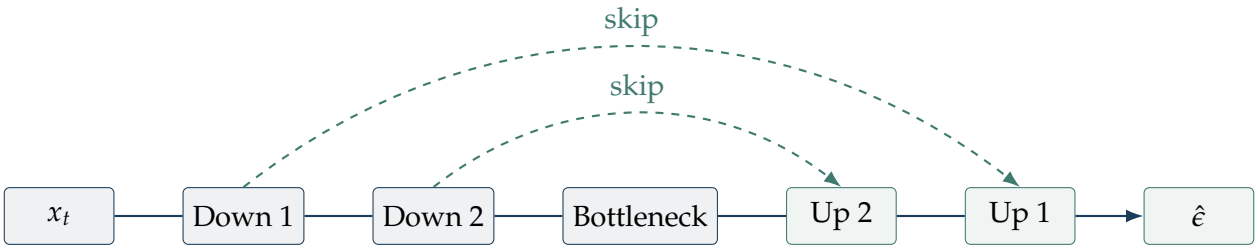
4.2 Pourquoi un U-Net ?

Pour les images, $\epsilon_\theta(x_t, t)$ est souvent un U-Net : un chemin descendant extrait des caractéristiques globales, un chemin montant reconstruit la résolution, et des *skip connections* transmettent les détails fins.

QUEL EST LE RÔLE EXACT DE t DANS LE U-NET ?

Le réseau doit savoir **à quel niveau de bruit** se trouve x_t. L'instant t n'est donc pas seulement un indice de boucle : il encode le rapport signal/bruit correspondant au niveau de diffusion courant.

En pratique, t est transformé en un **embedding temporel** (souvent sinusoïdal, puis projeté par un petit réseau) qui est injecté dans plusieurs blocs du U-Net. Le même débruiteur peut ainsi adapter son comportement : à grand t, il retire un bruit important et reconstruit surtout la structure globale ; à petit t, il effectue des corrections plus fines.



condition temporelle t injectée dans le réseau

5 Génération progressive

Une fois le réseau entraîné, on part de

$$x_T \sim \mathcal{N}(0, I)$$

et on applique les transitions inverses de T vers 1. Une forme classique de la mise à jour DDPM est :

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z,$$

avec $z \sim \mathcal{N}(0, I)$ pour $t > 1$.

QUE VAUT σ_t DANS UN DDPM ?

Pour le processus direct, le posterior exact $q(x_{t-1} \mid x_t, x_0)$ possède la variance

$$\tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t.$$

Un choix théorique courant pour l'échantillonnage ancestral consiste donc à fixer

$$\boxed{\sigma_t^2 = \tilde{\beta}_t.}$$

Le DDPM original discute aussi des choix de variance fixée proches, notamment β_t. Des travaux ultérieurs apprennent ou paramètrent cette variance. Pour cette séance, nous utiliserons $\sigma_t^2 = \tilde{\beta}_t$ comme convention de référence.

DU CONCEPT AU CODE — BOUCLE DE GÉNÉRATION

```
x = torch.randn(shape, device=device)
for t in reversed(range(T)):
    x = p_sample(model, x, t)
```

À observer : contrairement à un GAN, la génération n'est pas un seul passage avant ; elle répète une correction de débruitage à chaque instant.

LIMITE PRINCIPALE

Cette génération progressive repose sur un entraînement généralement stable et offre souvent une bonne couverture des modes, mais elle peut nécessiter des centaines ou des milliers d'évaluations du réseau. Le coût d'échantillonnage est donc l'un des principaux défauts des DDPM classiques.

6 Accélération : DDIM et autres stratégies

6.1 DDIM

DDIM conserve le même modèle entraîné mais modifie la trajectoire d'échantillonnage. L'idée importante est qu'il construit une famille de **processus directs non markoviens** ayant les mêmes marginales $q(x_t \mid x_0)$ que le DDPM. Le même réseau de prédiction du bruit peut donc être réutilisé avec une autre trajectoire inverse.

POURQUOI PEUT-ON SAUTER DES ÉTAPES ?

Comme l'entraînement apprend à estimer x_0 (ou, de manière équivalente, le bruit) à partir de n'importe quel x_t, la mise à jour peut relier directement deux instants sélectionnés t et $s < t$ sans visiter tous les temps intermédiaires. La formule DDIM ci-dessous formalise cette propriété.

On **admet ici la dérivation complète** issue du processus non markovien. Il faut retenir que les marginales de bruit restent compatibles avec le modèle entraîné, ce qui permet de choisir un sous-ensemble d'instants lors de l'échantillonnage.

DDPM VS DDIM

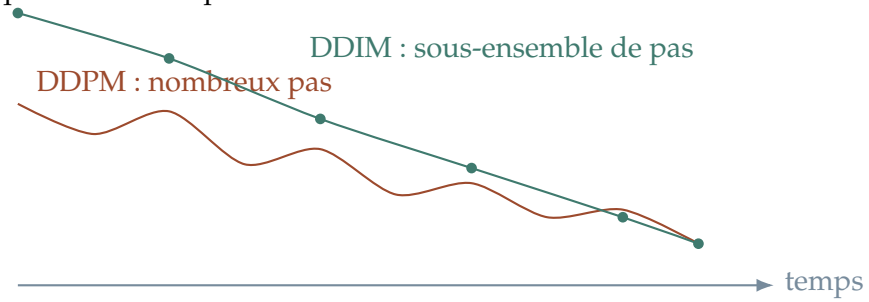
— **DDPM** : trajectoire stochastique, génération fidèle mais souvent longue ;

— **DDIM** : trajectoire pouvant être déterministe et sous-échantillonnée, donc génération plus rapide.

Une écriture générale de la mise à jour DDIM présentée dans le contenu source est :

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \left(\frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}} \right) + \sqrt{1-\bar{\alpha}_{t-1}-\sigma_t^2}\,\epsilon_\theta(x_t, t) + \sigma_t z.$$

Lorsque $\sigma_t = 0$, la trajectoire devient déterministe et permet de sauter des étapes. Dans la formulation DDIM, σ_t est un paramètre de stochasticité du sampler : le cas déterministe correspond classiquement à $\eta = 0$, tandis que des valeurs positives réintroduisent du bruit.



D'autres approches accélèrent aussi la génération : sous-échantillonnage temporel, solveurs numériques plus efficaces, distillation progressive et consistency models.

— **Sous-échantillonnage temporel** : utiliser seulement un sous-ensemble des instants t ;

— **Solveurs numériques** : améliorer l'intégration du processus inverse ;

— **Distillation progressive** : apprendre à franchir plusieurs étapes à la fois ;

— **Consistency models** : viser des mappings beaucoup plus directs du bruit vers les données.



7 Généralisation multi-modalité

Les modèles de diffusion ne sont pas limités aux images. L'idée générale — bruiter puis apprendre à débruiter — peut être adaptée à de nombreux espaces.

Modalité	Adaptation	Exemples d'usage
Image	Diffusion dans l'espace pixel ou latent	génération, inpainting, super-résolution
Texte	embeddings continus, latents continus ou diffusion discrète	génération / édition de séquences
Audio	diffusion sur onde, spectrogramme ou latent	parole, musique, débruitage
Conditionnelle	injection d'une condition y dans le débruiteur	classe, texte-à-image, image-à-image

7.1 Texte et audio

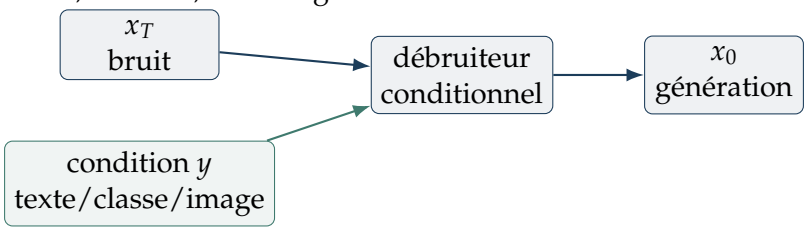
Pour le texte, trois stratégies sont courantes dans le contenu source : diffusion dans l'espace continu des embeddings, diffusion dans un espace latent continu, ou diffusion directement adaptée aux variables discrètes. Pour l'audio, la diffusion est utilisée pour la génération vocale, le débruitage, la super-résolution et la synthèse musicale.

7.2 Diffusion conditionnelle

On modélise alors

$$p_\theta(x_{t-1} \mid x_t, y),$$

où y peut être une classe, un texte, une image ou toute autre information de contrôle.



7.3 Guidage sans classificateur

Pour une génération conditionnelle moderne, on souhaite souvent renforcer l'influence de la condition y (classe, texte, etc.) sans entraîner un classificateur externe. Le **guidage sans classificateur** (*classifier-free guidance*) entraîne le même débruiteur à fonctionner parfois avec la condition et parfois sans condition.

À l'échantillonnage, on combine les deux prédictions :

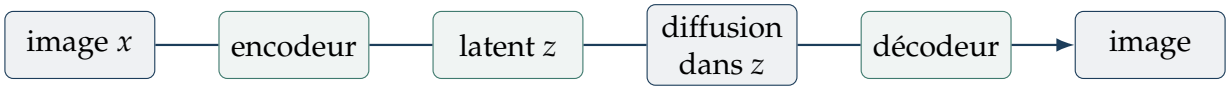
$$\hat{\epsilon}_{\text{guidé}} = \epsilon_\theta(x_t, t, \varnothing) + w\left[\epsilon_\theta(x_t, t, y) - \epsilon_\theta(x_t, t, \varnothing)\right],$$

où w contrôle la force du guidage.

> INTERPRÉTATION DE w
>
> Quand w augmente, la génération suit généralement plus fortement la condition, au prix possible d'une réduction de diversité ou d'artefacts si le guidage devient excessif. Le guidage sans classificateur est devenu un mécanisme central de nombreux systèmes texte-à-image modernes.

7.4 Diffusion latente : principe

Diffuser directement dans l'espace des pixels est coûteux pour les images haute résolution. Les **modèles de diffusion latente** déplacent le processus de diffusion dans un espace compressé appris par un autoencodeur.

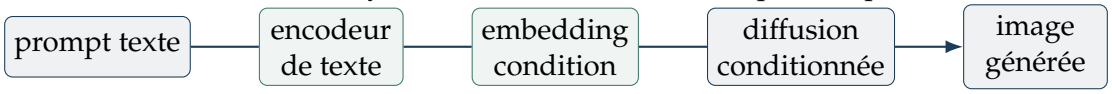


Le gain vient du fait que le débruiteur travaille sur une représentation spatiale plus petite que l'image originale. Dans les systèmes texte-à-image, le latent peut en plus être conditionné par une représentation du texte, par exemple via de l'attention croisée.

> PORTÉE DANS CE COURS
>
> La S12 introduit seulement le **principe** de la diffusion latente comme extension importante des modèles de diffusion. Son rôle dans les architectures hybrides VAE + diffusion et les systèmes génératifs modernes sera repris plus en détail en S14.

7.5 Texte-à-image : vue conceptuelle

Les systèmes texte-image modernes combinent typiquement un encodeur de texte et un modèle de diffusion conditionné par la représentation du prompt. Des exemples historiques importants incluent DALL-E 2, Imagen et Stable Diffusion, qui combinent un encodeur de texte avec un modèle de diffusion conditionné ; certains systèmes utilisent aussi une étape de super-résolution.



8 Comparaison et perspectives

Modèle	Qualité	Diversité	Entraînement	Échantillonnage
GAN	Souvent élevée	Variable	Peut être instable	Très rapide
VAE	Variable	Souvent bonne	Généralement stable	Rapide
Flow	Variable	Variable	Généralement stable	Rapide
AR	Variable	Dépend du décodage	Généralement stable	Lent
Diffusion	Souvent élevée	Souvent bonne	Généralement stable	Lent

Les orientations présentées dans le contenu source concernent notamment l'accélération de l'échantillonnage, les modèles à grande échelle, le contrôle fin de la génération et des applications telles que la découverte de médicaments, le design de protéines et la génération 3D.

9 Références essentielles

Références

[1] J. Ho, A. Jain, P. Abbeel, *Denoising Diffusion Probabilistic Models*, NeurIPS, 2020.
[2] J. Song, C. Meng, S. Ermon, *Denoising Diffusion Implicit Models*, ICLR, 2021.
[3] A. Q. Nichol, P. Dhariwal, *Improved Denoising Diffusion Probabilistic Models*, ICML, 2021.
[4] Y. Song, S. Ermon, *Generative Modeling by Estimating Gradients of the Data Distribution*, NeurIPS, 2019.
[5] Y. Song et al., *Score-Based Generative Modeling through Stochastic Differential Equations*, ICLR, 2021.
[6] J. Ho, T. Salimans, *Classifier-Free Diffusion Guidance*, 2022.
[7] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, B. Ommer, *High-Resolution Image Synthesis with Latent Diffusion Models*, CVPR, 2022.

10 Synthèse de la semaine 12 et transition vers S13

CE QU'IL FAUT RETENIR

1. Le **processus direct** ajoute progressivement du bruit jusqu'à atteindre une distribution simple.
2. Le **processus inverse** apprend à retirer ce bruit étape par étape.
3. Dans un **DDPM**, le réseau apprend généralement à prédire le bruit ajouté ϵ.
4. L'objectif d'entraînement se réduit à une **MSE sur le bruit**, ce qui rend l'apprentissage stable.
5. La génération est **progressive** et donc plus lente que dans les modèles à génération directe.
6. DDIM et d'autres méthodes réduisent le nombre d'étapes nécessaires ; DDIM permet notamment des trajectoires déterministes et des sauts temporels.
7. Le schedule β_t contrôle l'évolution du rapport signal/bruit et la variance σ_t^2 détermine la stochasticité du processus inverse.
8. La classifier-free guidance renforce le contrôle conditionnel sans classificateur externe.
9. La diffusion latente réduit le coût en appliquant le processus de diffusion dans une représentation compressée.

TRANSITION VERS LA S13

La S12 a étudié une chaîne de variables $x_0, x_1, \ldots, x_T$ correspondant à différents niveaux de bruit. En S13, nous élargirons cette idée de hiérarchie : plusieurs niveaux latents ou plusieurs échelles peuvent représenter des structures différentes d'une même donnée. Nous passerons ainsi aux ***modèles hiérarchiques et à la génération multi-échelle****.*

AUTO-VÉRIFICATION

1. Quelle est la différence entre le processus direct et le processus inverse ?
2. Pourquoi peut-on produire directement x_t à partir de x_0 ?
3. Quelle quantité le réseau DDPM prédit-il le plus souvent ?
4. Pourquoi la loss DDPM peut-elle être écrite comme une MSE sur le bruit ?
5. Pourquoi la génération DDPM est-elle lente ?
6. Quelle différence conceptuelle y a-t-il entre DDPM et DDIM ?
7. Comment conditionner un modèle de diffusion avec un texte ou une classe ?
8. Quel est le rôle du schedule de β_t ?
9. Que représente σ_t^2 dans la transition inverse ?
10. Quel est le principe de la classifier-free guidance ?
11. Pourquoi effectuer la diffusion dans un espace latent peut-il réduire le coût de calcul ?